

ACTIVIDAD AUTÓNOMA

API DE CATEGORÍAS CON FASTAPI

Módulo III - Construcción y Pruebas de una API REST

Tecnologías	Python · FastAPI · Pydantic · pytest · TestClient
Modalidad	Trabajo autónomo individual
Duración sugerida	2 a 3 horas
Producto	API REST de categorías + suite de pruebas automatizadas

Propósito: Aplicar de manera autónoma los conceptos de FastAPI aprendidos en el Módulo III mediante la construcción y prueba de un CRUD de categorías.

1. Competencia a desarrollar

Construir y verificar una API REST básica con FastAPI, aplicando modelos Pydantic, validaciones, códigos HTTP, manejo de errores, filtros y pruebas automatizadas.

2. Contexto de la actividad

Una tienda tecnológica necesita administrar las categorías utilizadas para clasificar sus productos. Debes implementar un servicio REST denominado API de Categorías. En esta actividad no se utilizará una base de datos real: los datos se almacenarán temporalmente en memoria para concentrar el trabajo en el comportamiento de la API y sus pruebas.

3. Requerimiento funcional

Cada categoría debe manejar la siguiente información:

Campo	Tipo	Obligatorio	Regla principal
id	int	Generado	Identificador único
name	str	Sí	3 a 50 caracteres
description	str None	No	Máximo 200 caracteres
active	bool	No	True por defecto

4. Modelo JSON esperado

```
{
  "id": 1,
  "name": "Computadores",
  "description": "Equipos de cómputo",
  "active": true
}
```

5. Reglas de validación

- name es obligatorio y debe tener entre 3 y 50 caracteres.
- description es opcional y puede tener como máximo 200 caracteres.
- active debe ser booleano y su valor por defecto será true.
- id será generado por la aplicación; el cliente no debe definirlo al crear una categoría.
- Cuando una categoría no exista, la API deberá responder con código HTTP 404 y un mensaje claro.
- Cuando Pydantic rechace datos inválidos, se espera el código HTTP 422.

6. Endpoints obligatorios

Método	Endpoint	Función	Código exitoso
GET	/categories	Listar categorías	200
GET	/categories/{category_id}	Consultar una categoría	200
POST	/categories	Crear categoría	201
PATCH	/categories/{category_id}	Actualizar parcialmente	200
DELETE	/categories/{category_id}	Eliminar categoría	204
GET	/categories?active=true	Filtrar categorías activas	200

7. Estructura mínima del proyecto

```
actividad-categorias/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── schemas.py
│   └── database.py
├── tests/
│   ├── __init__.py
│   └── test_categories.py
├── requirements.txt
└── README.md
```

8. Desarrollo de la actividad

1. Crea la estructura del proyecto y un entorno virtual de Python.
2. Instala FastAPI, Uvicorn, pytest y httpx.
3. Crea los modelos CategoryCreate, CategoryUpdate y Category utilizando Pydantic.
4. Define una lista categories_db con al menos dos categorías iniciales.
5. Implementa GET /categories y el filtro opcional active.
6. Implementa GET /categories/{category_id} y genera 404 cuando el recurso no exista.
7. Implementa POST /categories y devuelve 201 al crear correctamente.
8. Implementa PATCH /categories/{category_id} utilizando actualización parcial.
9. Implementa DELETE /categories/{category_id} y devuelve 204 cuando elimine correctamente.
10. Prueba manualmente todos los endpoints desde Swagger en /docs.
11. Crea las pruebas automatizadas solicitadas con pytest y TestClient.
12. Ejecuta la suite completa con pytest -v y corrige todos los fallos antes de entregar.

9. Casos de prueba mínimos

ID	Escenario	Petición / condición	Esperado
CA01	Listar categorías	GET /categories	200 y lista JSON
CA02	Consultar existente	GET /categories/1	200
CA03	Consultar inexistente	GET /categories/999	404
CA04	ID inválido	GET /categories/abc	422
CA05	Crear válida	POST con datos correctos	201
CA06	Nombre demasiado corto	POST con name de 1-2 caracteres	422
CA07	Falta nombre	POST sin name	422
CA08	Actualizar existente	PATCH /categories/1	200
CA09	Actualizar inexistente	PATCH /categories/999	404
CA10	Eliminar existente	DELETE /categories/1	204
CA11	Eliminar inexistente	DELETE /categories/999	404
CA12	Filtrar activas	GET /categories?active=true	200 y solo active=true

10. Requisitos de las pruebas automatizadas

- Utilizar fastapi.testclient.TestClient.
- Organizar las pruebas en tests/test_categories.py.
- Comprobar tanto status_code como datos relevantes de response.json().
- Incluir pruebas positivas y negativas.

- Evitar que una prueba contamine a otra. Se recomienda una fixture que restablezca `categories_db` antes de cada prueba.
- La entrega debe tener como mínimo 10 pruebas; se recomienda implementar las 12 de la matriz anterior.

11. Evidencias de entrega

- Repositorio o carpeta del proyecto con el código fuente.
- Archivo `requirements.txt`.
- `README.md` con instrucciones para instalar, ejecutar la API y correr las pruebas.
- Captura de Swagger mostrando los endpoints.
- Captura o evidencia de `pytest -v` con la suite ejecutada.
- Matriz de casos de prueba diligenciada con resultado obtenido: PASS o FAIL.

12. Criterios de evaluación

Criterio	Descripción	Peso
Estructura	Organización correcta del proyecto y dependencias.	10%
Modelos Pydantic	Create/Update/Response con tipos y restricciones correctas.	15%
CRUD	Endpoints GET, POST, PATCH y DELETE funcionando.	25%
Validaciones y errores	Uso correcto de 201, 204, 404 y 422.	15%
Filtro active	Query parameter implementado correctamente.	10%
Pruebas automatizadas	Casos positivos/negativos, asserts e independencia.	20%
Documentación	README y evidencias completas.	5%

13. Condiciones de aprobación

- La aplicación inicia correctamente con Uvicorn.
- Swagger funciona y muestra todos los endpoints obligatorios.
- El CRUD cumple con los códigos HTTP definidos.
- Los datos inválidos son rechazados por las validaciones.
- Los recursos inexistentes responden 404.
- El filtro active funciona.
- La suite de pruebas se ejecuta correctamente y no depende del orden de ejecución.

Importante: Esta actividad es autónoma. El aprendiz debe construir la solución a partir de los conceptos trabajados en clase. No se requiere base de datos externa; el énfasis está en FastAPI, validación, HTTP y pruebas automatizadas.

14. Reto opcional

Para obtener práctica adicional, agrega búsqueda por nombre:

```
GET /categories?search=comp
```

La respuesta deberá incluir únicamente las categorías cuyo nombre contenga el texto recibido, sin distinguir entre mayúsculas y minúsculas. Agrega al menos dos pruebas automatizadas para esta funcionalidad.

15. Comandos de referencia

```
python -m venv .venv

# Windows
.venv\Scripts\activate

# Linux/macOS
source .venv/bin/activate

pip install fastapi "uvicorn[standard]" pytest httpx
uvicorn app.main:app --reload
pytest -v
```

FIN DE LA ACTIVIDAD AUTÓNOMA